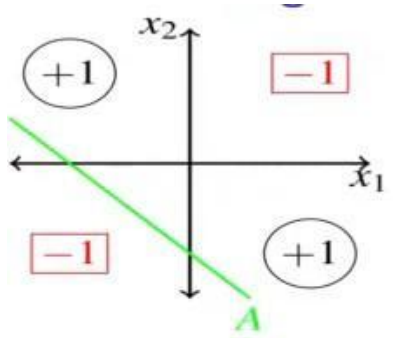
UNIDAD 3 — Redes Neuronales 2

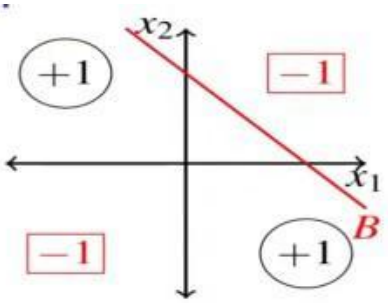
Perceptrón Multicapa

¿Cómo podríamos resolver el problema del XOR?

→ Lo podemos resolver con 3 perceptrones simples:

Primera capa: **perceptrones A y B**



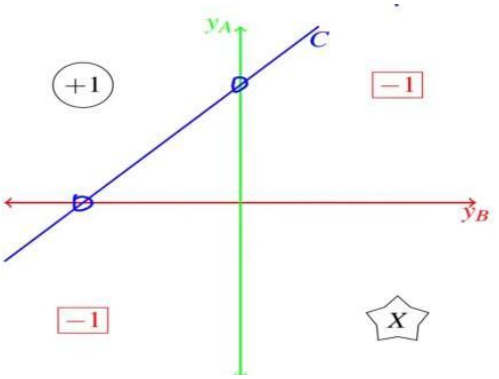


En la zona donde A da 1 y B da -1, deberíamos tener un perceptrón C que junte esas 2 salidas

Segunda capa: **perceptrón C**

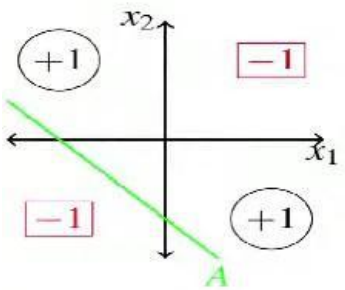
Vamos a considerar las salidas de A y B, $\mathbf{Y_A}$ e $\mathbf{Y_b}$ como entradas de C, y lo vamos a diseñar según la siguiente tabla de verdad:

y_A	y_B	y_C
+1	+1	−1
+1	−1	+1
−1	−1	−1



y el caso de $\mathbf{Y_A}$ = -1 e $\mathbf{Y_b}$ = +1 no va a darse nunca, entonces la región X no importa.

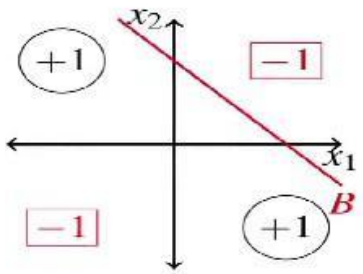
Combinación de perceptrones simples



Perceptrón A: $x_2 = -1 - x_1 = \frac{w_{A0}}{w_{A2}} - \frac{w_{A1}}{w_{A2}} x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{A0} = -1 \\ w_{A1} = +1 \\ w_{A2} = +1 \end{array} \right\} \rightarrow y_A = \text{sgn}(x_2 + x_1 + 1)$$

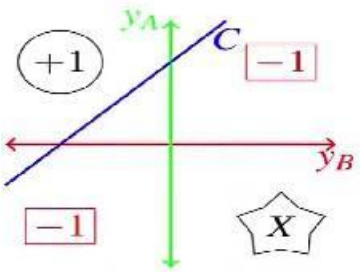
→ A suma las 2 entradas y a eso le suma 1.



Perceptrón B: $x_2 - +1 - x_1$

$$\rightarrow \left\{ \begin{array}{l} w_{B0} = +1 \\ w_{B1} = +1 \\ w_{B2} = +1 \end{array} \right\} \rightarrow y_B = \mathrm{sgn}(x_2 + x_1 - 1)$$

→ B suma las 2 entradas y a eso le resta 1.

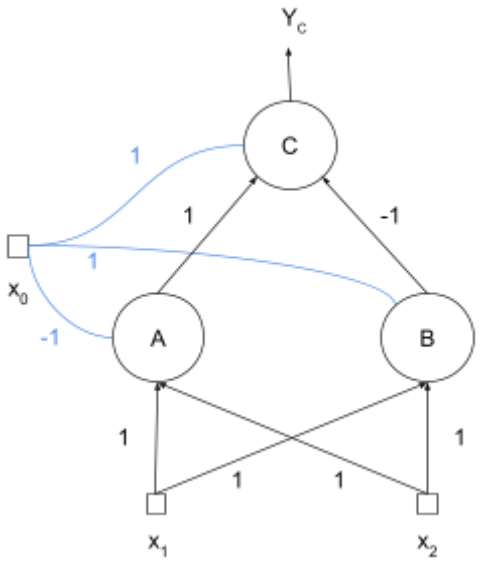


Perceptrón C: $y_A = +1 + y_B$

$$\rightarrow \left\{ \begin{array}{l} w_{C0} = +1 \\ w_{C1} = -1 \\ w_{C2} = +1 \end{array} \right\} \rightarrow y_C = \mathrm{sgn}(y_A - y_B - 1)$$

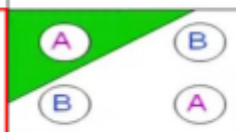
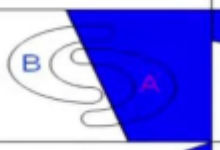
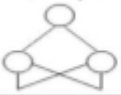
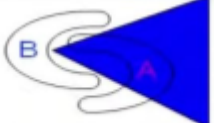
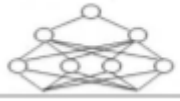
→ C resta Y_A menos Y_B y a eso le resta 1.

Arquitectura de esta red neuronal

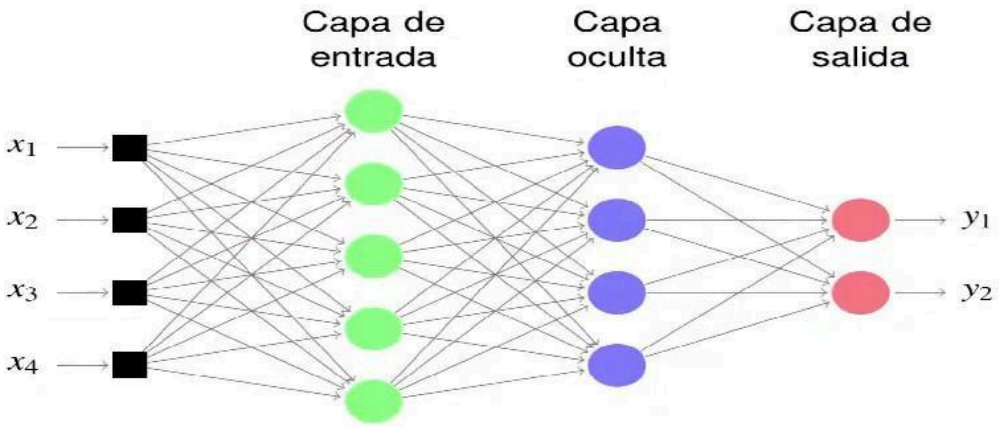


$$\left. \begin{array}{l} y_A = \mathrm{sgn}(x_2 + x_1 + 1) \\ y_B = \mathrm{sgn}(x_2 + x_1 - 1) \end{array} \right\} \rightarrow y_C = \mathrm{sgn}(y_A - y_B - 1)$$

Perceptrón Multicapa: Regiones de decisión y arquitectura

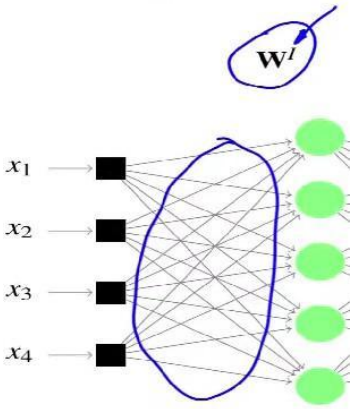
Estructura	Tipos de regiones de decisión	Problema XOR	Separación en clases	Formas regiones más generales
Una capa 	hemiplano limitado por hiperplano			
Dos capas 	Regiones convexas abiertas o cerradas			
Tres capas 	Arbitrarias (Complejidad limitada por N°. de Nodos)			

Arquitectura del perceptrón multicapa

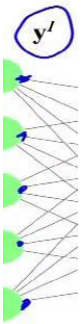


- En este caso tenemos datos de entrada en R^4
- Faltaría la entrada x_0 en el gráfico

→ **Notación vectorial:**



- Vamos a tener <u>MATRICES DE PESOS</u> **W** →
  (de tamaño: entradas x cant. percep. prox. capa)
- y <u>VECTORES DE SALIDAS</u> **y** →



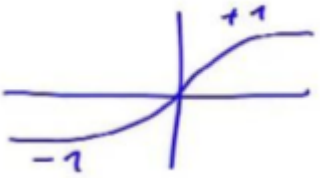
- Usamos superíndices para indicar el número de capa en el que estamos.

Cálculo de las salidas en cada capa (Propagación hacia adelante)

- Capa I:

$$v_j^I = \left\langle \mathbf{w}_j^I, \mathbf{x} \right\rangle = \sum_{i=0}^{N} w_{ji}^I x_i \quad \text{(completo } \mathbf{v}^I = \mathbf{W}^I \mathbf{x}\text{)}$$

$$y_j^I = \phi(v_j^I) = \frac{2}{1 + e^{-bv_j^I}} - 1 \quad \text{(simétrica } \perp 1\text{)}$$



→ en lugar de la función signo, usamos la función sigmoide →

- Capa II:

$$v_j^{II} = \left\langle \mathbf{w}_j^{II}, \mathbf{y}^I \right\rangle \quad \rightarrow \quad y_j^{II} = \phi(v_j^{II})$$

- Capa III:

$$v_j^{III} = \left\langle \mathbf{w}_j^{III}, \mathbf{y}^{II} \right\rangle \quad \rightarrow \quad y_j^{III} = \phi(v_j^{III}) = y_j$$

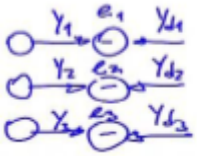
Propagación hacia atrás: caso general y capa de salida

→ Es lo que permite actualizar los pesos, hacer que la red neuronal aprenda iteración a iteración.

→ Definimos un **criterio de error**:

Suma del error cuadrático instantáneo

Es un error para el instante n, para la iteración n.



$$\xi(n) = \frac{1}{2} \sum_{j=1}^{M} e_j^2(n)$$

Restamos cada una de las salidas, elevamos al cuadrado, y luego sumamos.

→ (así siempre la suma va creciendo cuando hay un error)

Aplicación del gradiente (caso general)

Modificación de los pesos:

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

- En el sentido opuesto al gradiente del error.
- La **velocidad de aprendizaje** μ es qué tan rápido nos movemos sobre la superficie de error:
  - Si es muy grande: se va a aprender el ejemplo que le acabo de mostrar, pero olvidar todo lo que aprendió antes.

- Si es muy pequeña: voy a tener que mostrarle muchas más veces las cosas para que las aprenda

- El **gradiente de error** es la derivada parcial del criterio de error cuadrático que definimos con respecto a los **pesos** w_{ij}.

  Tenemos que derivar a una función de una función de una función:
  - El criterio de error cuadrático medio **ξ** depende del error cuadrático medio e_j de la neurona asociada al peso **j.**
  - Ese error e_j depende de la salida no lineal y_j que haya dado esa neurona (la salida después de pasar por la función de activación.
  - Esa salida y_j depende de la salida lineal v_j (la salida antes de pasar por la función de activación).
  - Esa salida v_j es la que finalmente depende de los pesos w_{ij}.

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)} \frac{\partial v_j(n)}{\partial w_{ji}(n)}$$

$$\frac{\partial v_j(n)}{\partial w_{ji}(n)} = \frac{\partial \sum_{i=0}^{N} w_{ji}(n) y_i(n)}{\partial w_{ji}(n)} = y_i(n)$$

→ Observamos que para todos los valores de i, salvo cuando i valga el i de $w_{ij}(n)$, tenemos valores constantes respecto de lo que estamos derivando (sería la variable $w_{ij}(n)$).

Entonces nos quedará la dd de una constante ($y_i(n)$) por la variable ($w_{ij}(n)$)

OBS → y_i , con subíndice **i**, es la **ENTRADA** a la neurona, que fue pesada con el peso w_i.

→ y_j , con subíndice **j**, es la SALIDA no lineal de la neurona.

=> la modificación nos queda:

$$\frac{\partial \xi(n)}{\partial w_{ji}(n)} = \boxed{\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}} y_i(n)$$

Para seguir simplificando la expresión, definimos:

$$\text{Gradiente de error local instantáneo: } \delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{\partial y_j(n)}{\partial v_j(n)}$$

- "Local" → porque es de la neurona **j**.
- "Instantáneo" → porque está calculado en un instante **n** de esta iteración.
- Es una parte del gradiente global.

- Es una versión simplificada del gradiente respecto de la salida, a diferencia de la primera expresión donde estamos pasando a través del error $\mathbf{e_j}$.

→ Si analizamos la segunda parte del $\boldsymbol{\delta_j}$:

$$\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \boxed{\frac{\partial y_j(n)}{\partial v_j(n)}}$$

→ Es la derivada de la función de activación $\mathbf{y_j}$. (simétrica)

- **Desarrollo de la derivada de la sigmoidea:**

$$\begin{aligned}
\boxed{\frac{\partial y_j(n)}{\partial v_j(n)}} &= \frac{\partial \left\{ \frac{2}{1+e^{-v_j(n)}} - 1 \right\}}{\partial v_j(n)} \\
&= 2\frac{e^{-v_j(n)}}{(1+e^{-v_j(n)})^2} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \frac{e^{-v_j(n)}}{1+e^{-v_j(n)}} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \frac{\overbrace{-1+1}^{0} + e^{-v_j(n)}}{1+e^{-v_j(n)}} \\
&= 2\frac{1}{1+e^{-v_j(n)}} \left(\frac{-1}{1+e^{-v_j(n)}} + \frac{1+e^{-v_j(n)}}{1+e^{-v_j(n)}} \right)
\end{aligned}$$

$$\begin{aligned}
\frac{\partial y_j(n)}{\partial v_j(n)} &= 2\frac{1}{1+e^{-v_j(n)}} \left(1 - \frac{1}{1+e^{-v_j(n)}} \right) \\
&- 2\frac{y_j(n)+1}{2} \left(1 - \frac{y_j(n)+1}{2} \right) \\
&= (y_j(n)+1) \left(1 - \frac{y_j(n)+1}{2} \right) \\
&= (y_j(n)+1) \left(\frac{2 - y_j(n) - 1}{2} \right) \\
&= \boxed{\frac{1}{2}(y_j(n)+1)(y_j(n)-1)}
\end{aligned}$$

→ obtenemos una ecuación muy simple

=> volviendo a la definición del $\boldsymbol{\delta_j}$:

$$\delta_j = \frac{\partial \xi(n)}{\partial y_j(n)} \frac{1}{2}(1 + y_j(n))(1 - y_j(n))$$

→ Nos quedaría por calcular una única derivada, la cual va a depender de la capa en la que estemos parados

=> la modificación nos queda:

$$\boxed{\Delta w_{ji}(n) = \mu \delta_j(n) y_i(n)}$$

Propagación hacia atrás en la capa III (salida)

La modificación de los pesos de la última capa es:

$$\Delta w_{ji}^{III}(n) = \mu \delta_j^{III}(n) y_i^{II}(n)$$

Tenemos que calcular el gradiente de error instantáneo δ_j^{III}.

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

- Calculamos la derivada:

$$\delta_j^{III}(n) = -\frac{\partial \xi(n)}{\partial e_j(n)} \frac{\partial e_j(n)}{\partial y_j^{III}(n)} \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

  - Reemplazamos la expresión de **ξ,** y de $\mathbf{e_j}$:

$$\delta_j^{III}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_j e_j^2(n) \right\}}{\partial e_j(n)} \cdot \frac{\partial \left\{ d_j^{III}(n) - y_j^{III}(n) \right\}}{\partial y_j^{III}(n)} \cdot \frac{1}{2}(1 + y_j^{III}(n))(1 - y_j^{III}(n))$$

  - Derivamos:

$$\delta_j^{III}(n) = \frac{1}{2} e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n)) \bigstar$$

=> la modificación queda:

$$\Delta w_{ji}^{III}(n) = \eta e_j(n)(1 + y_j^{III}(n))(1 - y_j^{III}(n)) y_i^{II}(n)$$

(η absorbe todas las constantes que hay por ahí, y la velocidad de aprendizaje)

ΔW_{ji}^{III} = eta * error * dd de la f. de activación * entrada

Propagación hacia atrás en la capa II (oculta)

- Para calcular la derivada dentro del δ_j^{II} en este caso, vemos que <u>al error **ξ** lo estamos midiendo en la capa de salida</u> ($\mathbf{e_j}$ depende de $\mathbf{y_j}$), pero ahora queremos derivar con respecto a la capa oculta, osea que tenemos que atravesar ambas para hacerlo:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^{I}(n)$$

$$\delta_j^{II}(n) = -\frac{\partial \xi(n)}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\frac{\partial \left\{ \frac{1}{2} \sum_k e_j^2(n) \right\}}{\partial y_j^{II}(n)} \frac{1}{2}(1 + y_j^{II}(n))(1 - y_j^{II}(n))$$

y_j^{III}

(llamamos **k** al índice que recorre las neuronas de la capa de salida (III), y **j** es el de la capa actual (II), e **i** recorre a las salidas de la capa anterior (I))

- La dd de la sumatoria es la sumatoria de las dd:

$$\delta_j^{II}(n) = -\frac{1}{2}\sum_k \frac{\partial e_k^2(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n)\frac{\partial e_k(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

- Ahora sí vamos a recorrer todo lo que hay entre el error en la capa de salida y la neurona en la capa oculta contra la que queremos obtener la dd:

$$\delta_j^{II}(n) = -\sum_k e_k(n)\frac{\partial e_k(n)}{\partial y_k^{III}(n)}\frac{\partial y_k^{III}(n)}{\partial v_k^{III}(n)}\frac{\partial v_k^{III}(n)}{\partial y_j^{II}(n)}\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

$$\delta_j^{II}(n) = -\sum_k e_k(n)\cdot\frac{\partial\{d_k^{III}(n)-y_k^{III}(n)\}}{\partial y_k^{III}(n)}\cdot\frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n))\cdot$$
$$\cdot\frac{\partial\left\{\sum_j w_{kj}^{III}y_j^{II}(n)\right\}}{\partial y_j^{II}(n)}\cdot\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

dd de la función de activación en la capa de salida, y en la capa oculta

$$\delta_j^{II}(n) = \sum_k e_k(n)\cdot\frac{1}{2}(1+y_k^{III}(n))(1-y_k^{III}(n))\cdot w_{kj}^{III}\cdot$$
$$\cdot\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

  - Pero vemos que dentro de δ_j^{II} está metida la fórmula de δ_j^{III}, así que podemos reemplazarlo:

$$\delta_j^{II}(n) = \left[\sum_k \delta_k^{III}(n)w_{kj}^{III}\right]\frac{1}{2}(1+y_j^{II}(n))(1-y_j^{II}(n))$$

→ DE ESTA PARTE SURGE EL CONCEPTO DE "RETROPROPAGACIÓN":

- Tenemos un promedio ponderado del gradiente de error local de la capa III, lo que hace que **este gradiente pase a través de los pesos que unen la capa III con la capa II, de atrás para adelante.**

=> la modificación queda:

$$\Delta w_{ji}^{II}(n) = \mu \delta_j^{II}(n) y_i^I(n)$$

Por lo tanto:

$$\Delta w_{ji}^{II}(n) = \eta \left[\sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^I(n)$$

ΔW_{ji}^{III} = eta * retroprop. de g. de error de (III) hacia (II) * dd de la f. de activación (II) * entrada a (II)

Generalizando para la capa "p"

$$\Delta w_{ji}^{II}(n) = \eta \left[\sum_k \delta_k^{III} w_{kj}^{III}(n) \right] (1 + y_j^{II}(n))(1 - y_j^{II}(n)) y_i^I(n)$$

$$\Downarrow$$

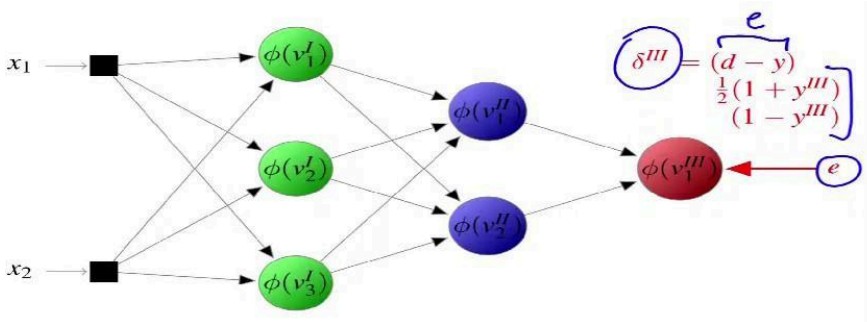
$$\Delta w_{ji}^{(p)}(n) = \eta \left\langle \delta^{(p+1)}, \mathbf{w}_j^{(p+1)} \right\rangle (1 + y_j^{(p)}(n))(1 - y_j^{(p)}(n)) y_i^{(p-1)}(n)$$

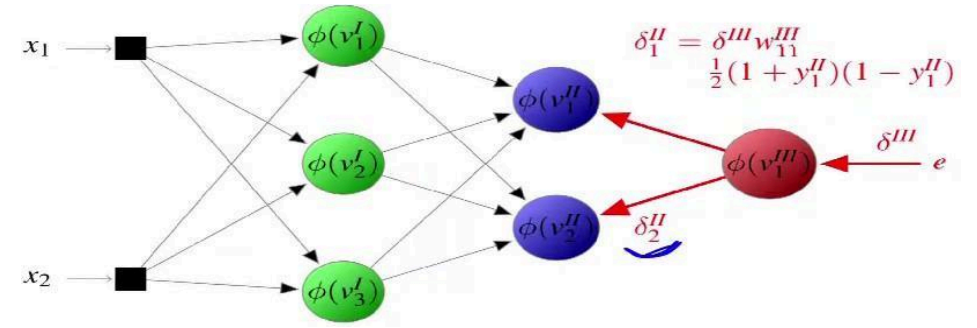
Resúmen del algoritmo de retropropagación (BP)

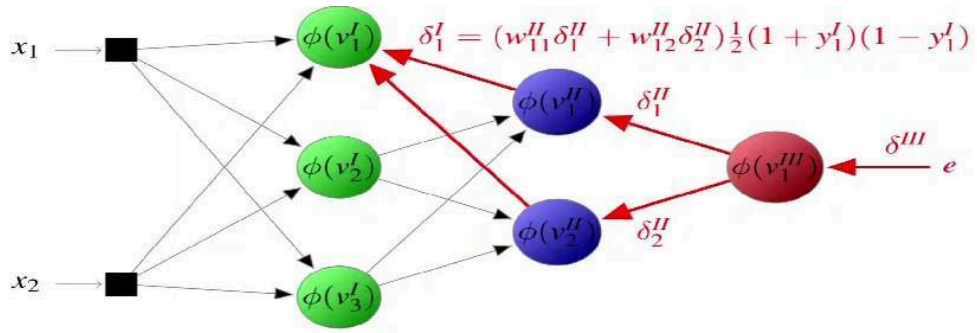
1. Inicialización aleatoria
2. Propagación hacia adelante
3. Propagación hacia atras
4. Adaptación de los pesos
5. Iteración: vuelve a 2 hasta convergencia o finalización

Repetimos (2-5) para todos los patrones (datos del archivo de entrenamiento), y a cada pasada con todos los datos se le llama **ÉPOCA**.

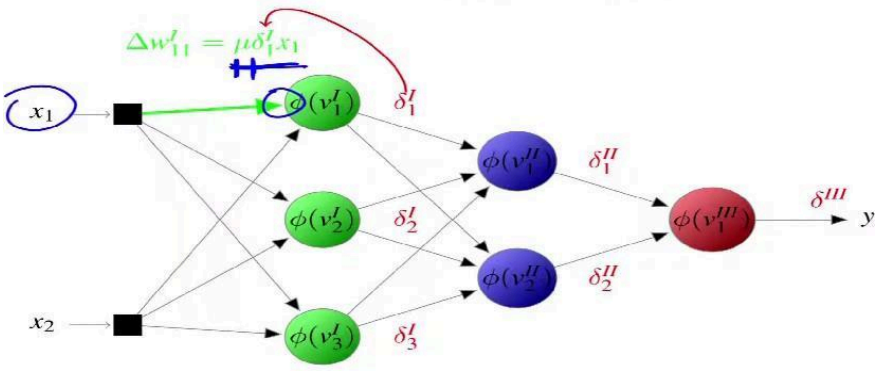
2.







3.

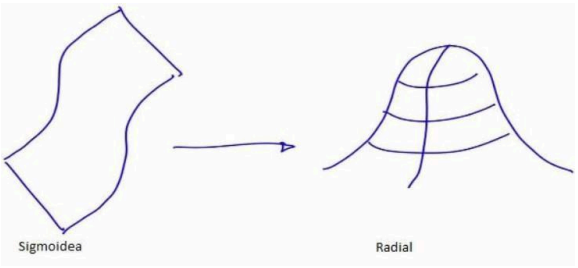


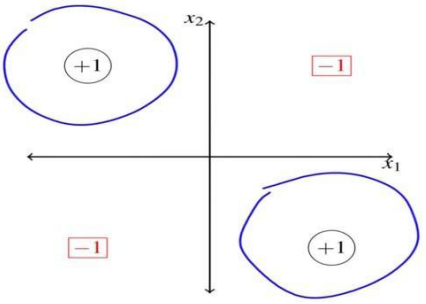
→ Aclaración: No importa el órden en que actualicemos los pesos, ya que ya tenemos calculadas las salidas de las neuronas.

Redes neuronales con funciones de base radial (RBF-NN)

El hecho de cambiar la función de activación nos permite cambiar la forma de las regiones de decisión, por ejemplo, en el caso del XOR:

Regiones radiales:





RBF: Funciones de base radial

Originalmente se usaban para aproximar funciones.

$$\phi : \mathbb{R}^N \rightarrow \mathbb{R}$$
$$d = \phi(\mathbf{x})$$

Aproximación:

$$h(\mathbf{x}) = \sum_j w_j \phi(\|\mathbf{x} - \boldsymbol{\mu}_j\|)$$

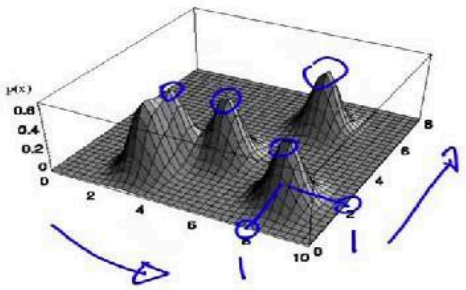
- Pesos $\mathbf{w}_j$ (equivalentes a los pesos sinápticos)
- Alguna función de base radial ϕ por la que se pasa una distancia entre nuestro patrón de entrada y un vector de medias $\boldsymbol{\mu}$.

- Vamos a usar esta **<u>FUNCIÓN GAUSSIANA</u>:**

$$\boldsymbol{\phi}(k) = e^{\left(\frac{-k^2}{2\sigma^2}\right)}$$

Aproximación: ejemplo

$$p(\mathbf{x}) = \sum_j w_j \phi(\|\mathbf{x} - \boldsymbol{\mu}_j\|)$$



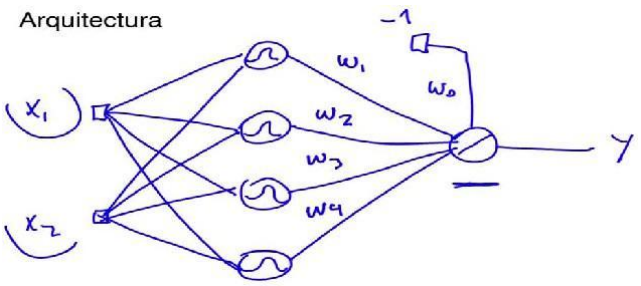
$\mu = \begin{bmatrix} 8 \\ 2 \end{bmatrix}$ → MEDIA mu

RBF-NN: generalidades

- Arquitectura: ——————————————————

  - ***Capa radial:***
    - 1er capa
    - Puede tener muchas neuronas
    - Aquí están las **gaussianas** (en el ejemplo, en R2)
      - Cada una tiene un vector de medias $\boldsymbol{\mu}$ distinto
      - El valor de $\boldsymbol{\sigma}$ para todas
    - Tiene sus pesos sinápticos que conectan a las entradas con ellas
    - NO tienen sesgo/bias. (no tienen w_0)
    - En general se ponen todos pesos = 1.

  - ***Perceptrón de la salida final:***
    - Último perceptrón con una función lineal
    - Una constante que escala la salida
    - Tiene sus pesos sinápticos + el peso $\mathbf{w_0}$
    - Es un perceptrón simple con salida lineal



Como las entradas a las radiales se le ponen pesos todos en 1, se ajusta la relación entre las entradas y la gaussiana con los parámetros de la gaussiana, la *media* $\boldsymbol{\mu}$ y la *desviación estándar* $\boldsymbol{\sigma}$.

- Modelo matemático: ——————————————————

$$y_k(\mathbf{x}_\ell) = \sum_{j=1}^{M} w_{kj}\phi_j(\mathbf{x}_\ell)$$

- **k** → recorre las salidas (pueden ser varias)
- **j** → recorre las neuronas de la capa radial
- $\mathbf{w_{kj}}$ → los w de la capa de salida (los otros son todos 1)
- ϕ_j → función de base radial
- $\mathbf{x_l}$ → la entrada

Donde: $\phi_j(x_l) = e^{(-||x_l - \mu_j||^2 / 2\sigma_j^2)}$

→ Para cada **j** tenemos un vector $\boldsymbol{\mu_j}$ distinto

RBF-NN: entrenamiento

Tiene 2 ETAPAS, un entrenamiento mixto:

- **Método 1:**
  - Adaptación no supervisada de las RBF
    - Utilizando el **método k-medias**
    - Utilizando mapas autoorganizativos
    - Otros…
  - Adaptación supervisada de los w_{kj} (LMS)
- **Método 2:**
  - Inicialización por el Método 1
  - Adaptación supervisada de las RBF $\left(\frac{\partial \xi}{\partial \mu_{ji}}, \frac{\partial \xi}{\partial \sigma_j}\right)$ → ajuste de los parámetros de la red

→ En general se adaptan RBF y w_{kj} por separado.

¿Cómo entrenar la capa radial? — ENTRENAMIENTO PARTE 1

- Método **NO-supervisado**! (uno de los más simples)
- Se entrena *SIN NECESIDAD DE CONOCER LA SALIDA CORRECTA*

→ Adaptación de las RBF: k-medias

→ Objetivos:

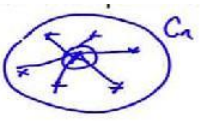
- Encontrar **k** (uno para cada neurona) conjuntos $\mathbf{C_j}$ de patrones de entrada de forma que:
  - Cada conjunto $\mathbf{C_j}$, sea *lo más **diferente*** posible de los demás
  - Los patrones $\mathbf{x_l}$ dentro de cada $\mathbf{C_j}$ sean *lo más **parecidos*** posible entre ellos
- Encontrar el centroide μ_j de cada conjunto $\mathbf{C_j}$ → luego serán los de las gaussianas en la neurona **j** correspondiente

=> lo que tenemos que obtener son los vectores μ_j , en la dimensión que corresponda, que van a ser las medias de las gaussianas ϕ_j , y el promedio de todos los patrones que hayan quedado en los conjuntos $\mathbf{C_j}$

Ecuación de optimización: $\min \left\{ J = \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\}$

→ Minimizar CRITERIO J, compuesto por:

- La distancia entre los patrones que corresponden al conjunto $\mathbf{C_j}$ menos el vector de medias μ de ese conjunto.
- Vamos acumulando todas las distancias anteriores de cada conjunto.



- Hacemos luego una sumatoria de eso para todos los conjuntos.

A. Adaptación de las RBF: k-medias *por lotes*

1. **Inicialización:** se forman los **k** conjuntos $\mathbf{C_j(0)}$ con patrones $\mathbf{x_l}$ elegidos aleatoriamente.
2. **Se calculan los centroides**: → hacemos un promedio (suma de vectores)

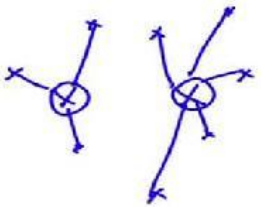
$$\boldsymbol{\mu}_j(n) = \frac{1}{|C_j(n)|} \sum_{\ell \in C_j(n)} \mathbf{x}_\ell$$

3. **Se reasignan los patrones x_l al conjunto C_j más cercano**:

$$\ell \in C_j(n) \Leftrightarrow \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 < \|\mathbf{x}_\ell - \boldsymbol{\mu}_i\|^2 \quad \forall i \neq j$$

el l-ésimo patrón pertenece al conjunto j, sí y solo sí la distancia entre ellos es menos que la distancia entre el patrón y el centroide de todos los demás conjuntos posibles.

Entonces =>
→ Hacemos que cada conjunto tome a los patrones que tiene más cerca, o
→ que cada patrón se vaya al conjunto cuyo centroide tenga más cerca.



4. Volver a **2** hasta que no se realicen reasignaciones.

B. Adaptación de las RBF: k-medias *online*

→ Ajuste patrón a patrón, uno a uno:

Dado un patrón intentamos minimizar este criterio general:

Optimización por método de gradiente:

$$\nabla_\mu J = \nabla_\mu \left\{ \sum_{j=1}^{k} \sum_{\ell \in C_j} \|\mathbf{x}_\ell - \boldsymbol{\mu}_j\|^2 \right\} = 0$$

y obtenemos una regla:

$$\boldsymbol{\mu}_j(n+1) = \boldsymbol{\mu}_j(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_j(n))$$

1. **Inicialización:** se eligen **k** patrones aleatoriamente y se usan como centroides iniciales $\boldsymbol{\mu}_j(0) = \mathbf{x}'$ → un patrón al azar.

2. **Selección del centroide ganador:**

$$j* = \arg\min_j \{\|\mathbf{x}_\ell - \boldsymbol{\mu}_j(n)\|\}$$

3. **Adaptación:**

$$\boldsymbol{\mu}_{j*}(n+1) = \boldsymbol{\mu}_{j*}(n) + \eta(\mathbf{x}_\ell - \boldsymbol{\mu}_{j*}(n))$$

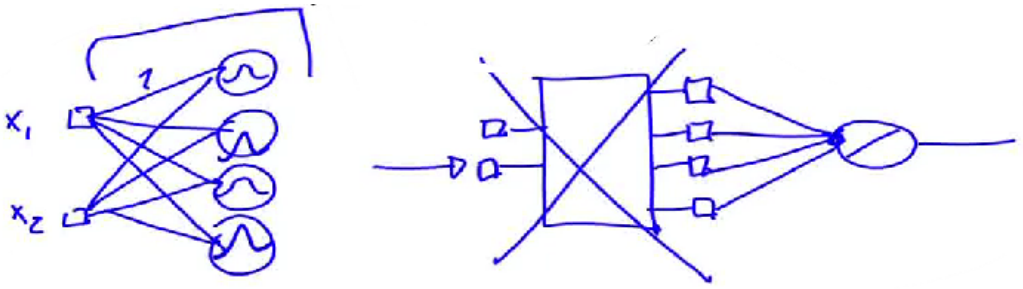
4. **Volver a 2** (por cada patrón) **hasta no encontrar mejoras significativas en J.**

¿Cómo entrenar la capa de salida? — ENTRENAMIENTO PARTE 2

- SÍ ES SUPERVISADO*
- Al entrenar los pesos, las RBF quedan fijas (dejamos fija la capa radial)
- Al estar las RBF fijas se pueden obtener las **salidas intermedias** para cada patrón de entrada:
- Con esas salidas intermedias, que las podemos considerar simplemente como entradas nuevas, se puede entrenar cada perceptrón simple, como ya sabemos:

$$\mathbf{y} = \mathbf{W}\boldsymbol{\phi}(\mathbf{x}_\ell)$$

→ estamos fijando el ϕ



→ **Métodos de entrenamiento para W:**

• pseudo-inversa del vector ϕ**(x$_l$)**

• gradiente descendiente sobre el error cuadrático instantáneo (**LMS**) →

Adaptación de los pesos: método LMS

Recordando la fórmula:

$$\Delta w_{ji}(n) = -\mu \frac{\partial \xi(n)}{\partial w_{ji}(n)}$$

- Medimos el error:

$$e_k(n) = y_k(n) - d_k(n)$$

* → necesitamos la salida correcta

k → de la cantidad de salidas (cantidad de neuronas en la capa de salida)

- Lo usamos para el error cuadrático medio:

$$\xi(n) = \frac{1}{2}\sum_k e_k^2(n) = \frac{1}{2}\sum_k \left(\sum_j w_{kj}(n)\phi_j(n) - d_k(n)\right)^2$$

- Calculamos la derivada:

$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = (y_k(n) - d_k(n)) \frac{\partial}{\partial w_{kj}} \left(\sum_j w_{kj}(n)\phi_j(n) - d_k(n) \right)$$

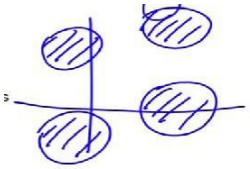
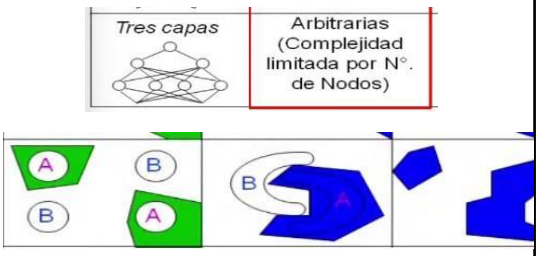
$$\frac{\partial \xi(n)}{\partial w_{kj}(n)} = e_k(n)\phi_j(n)$$ salida de la capa radial

Regla de aprendizaje:

$$w_{kj}(n+1) = w_{kj}(n) - \eta e_k(n)\phi_j(n)$$

$$w_{kj}(n+1) = w_{kj}(n) - \eta \left(\sum_i w_{ki}(n)\phi_i(n) - d_k(n) \right) \phi_j(n)$$

Comparación RBF-NN (redes con RBF) vs. MLP (perceptrones multicapa)

RBF-NN	MLP
Con una sola capa oculta puede resolver cualquier tipo de problemas 	Necesita *p* capas ocultas para problemas más complejos 
Tenemos solo las funciones de base radial	Tenemos que hacer combinaciones de hiperplanos sigmoideos
Representaciones locales, zonas acotadas del espacio, sumadas	Representaciones distribuidas, zonas no acotadas del espacio, combinadas
Convergencia más simple, no tenemos retropropagación	Tiene retropropagación
Arquitectura más simple	Arquitectura más compleja
Combinación de aprendizaje supervisado y no supervisado	Solo aprendizaje supervisado

Gaussianas N-dimensionales

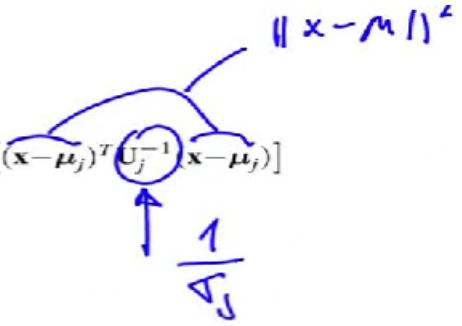
Concepto, interpretación gráfica y forma matricial:

- **x** → vector de entradas
- μ → vector de medias en R^N
- $\mathbf{U_j}$ → **matriz de covarianza** (Compuesto por las <u>desviaciones estándar</u>)
  - que modifican la forma de la campana

FUNCIÓN GAUSSIANA:

Forma general $\rightarrow \mathbf{x}, \boldsymbol{\mu}_j \in \mathbb{R}^N, \mathbf{U}_j \in \mathbb{R}^{N \times N}$:

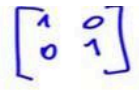
$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}|\mathbf{U}_j|^{1/2}} \cdot e^{-\frac{1}{2}[(\mathbf{x}-\boldsymbol{\mu}_j)^T \mathbf{U}_j^{-1} (\mathbf{x}-\boldsymbol{\mu}_j)]}$$

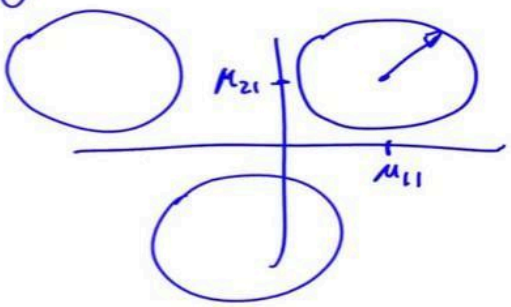


Análisis de distintos casos de U_j:

- **<u>U = matriz identidad</u>**
  - Caso simplificado 3 $\rightarrow \mathbf{U}_j = \mathbf{I}$:

$$\mathcal{N}'(\mathbf{x}, \boldsymbol{\mu}_j) = e^{-\frac{1}{2}\sum_{k=1}^{N}(x_k - \mu_{jk})^2}$$

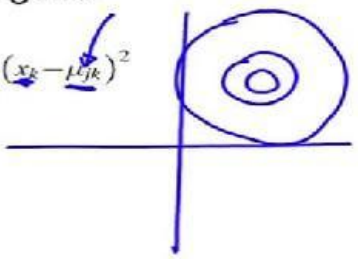




- σ → es = 1, fijo, no lo podemos modificar
- Como la varianza es 1 nos da un círculo, podemos tener varias neuronas pero cada una es un círculo, porque el desvió estándar al estar fijo no nos deja mover.
- Pero luego con los pesos en el perceptrón simple le podemos dar más o menos importancia, evitando así tener que calcular las desviaciones para modificar los círculos.
- Por lo general con este modelo se pueden resolver muchos problemas.

- **<u>U = diagonal con valores iguales, pero no necesariamente 1</u>**
  - Caso simplificado 2 $\rightarrow \mathbf{U}_j \in \mathbb{R}^{N \times N}$, diagonal igual:

$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}\sqrt{N}\sigma} \cdot e^{-\frac{1}{2\sigma^2}\sum_{k=1}^{N}(x_k - \mu_{jk})^2}$$



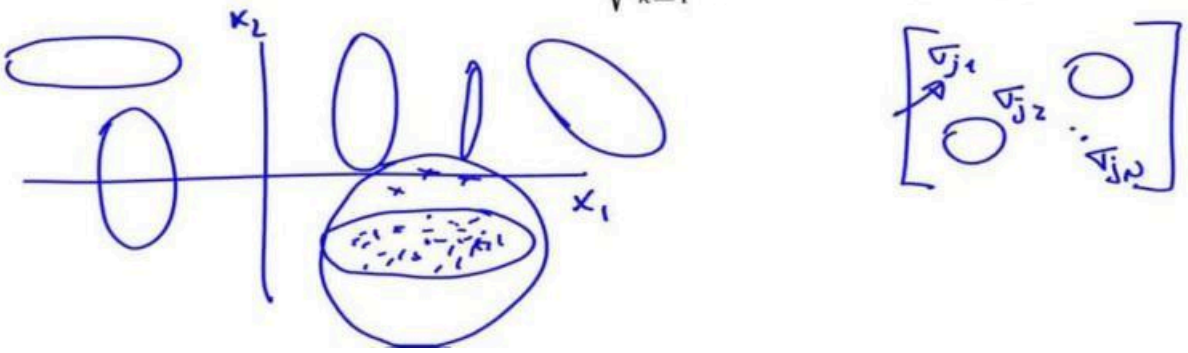
- σ → es un escalar, no depende de k ni j.
- Este nos permite modificar el σ, de manera que un σ grande permite un círculo grande y un σ pequeña un círculo pequeño

  => ¿Cómo calculo ese σ?
  - Si tengo el centroide calculado por el método de k medias, simplemente calculando las desviaciones de los patrones desde el centroide.

- **<u>U = diagonal con valores generales</u>**

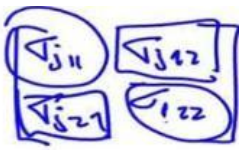
- Caso simplificado 1 → $\mathbf{U}_j \in \mathbb{R}^{N\times N}$, diagonal general:

$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}\sqrt{\sum_{k=1}^{N}\sigma_{jk}^2}} \cdot e^{-\frac{1}{2}\sum_{k=1}^{N}\frac{(x_k-\mu_{jk})^2}{\sigma_{jk}^2}}$$



- σ → depende de **k**, cada dimensión, y depende de **j**, de cada neurona
- <u>Esto nos permite generar elipses, no solo esferas, ya que cada dimensión tiene su varianza independiente de las otras.</u>
- DETALLE: → Solo las podemos poner en vertical u horizontal.

- **<u>FORMA GENERAL, U = matriz COMPLETA:</u>**
  - Podemos tener gaussianas inclinadas, ya que acá sí hay correlación entre las dimensiones, es decir que NO son independientes unas de otras.
  - Es el caso donde tendríamos la matriz COMPLETA:



- Forma general → $\mathbf{x}, \boldsymbol{\mu}_j \in \mathbb{R}^N, \mathbf{U}_j \in \mathbb{R}^{N\times N}$:

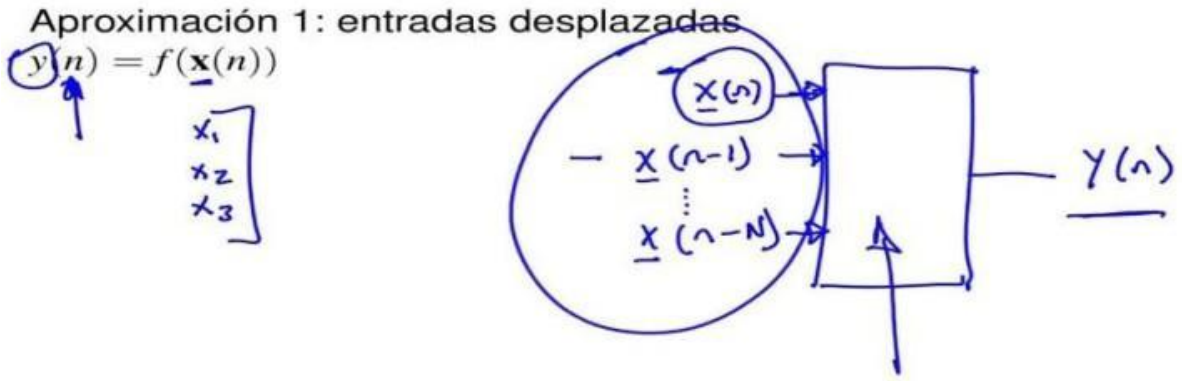
$$\mathcal{N}(\mathbf{x}, \boldsymbol{\mu}_j, \mathbf{U}_j) = \frac{1}{(2\pi)^{N/2}|\mathbf{U}_j|^{1/2}} \cdot e^{-\frac{1}{2}[(\mathbf{x}-\boldsymbol{\mu}_j)^T \mathbf{U}_j^{-1}(\mathbf{x}-\boldsymbol{\mu}_j)]}$$

Redes Neuronales Dinámicas (DNN)

→ Son redes que internamente tienen un comportamiento dinámico que evoluciona a lo largo del tiempo.

¿Por qué dinámica?

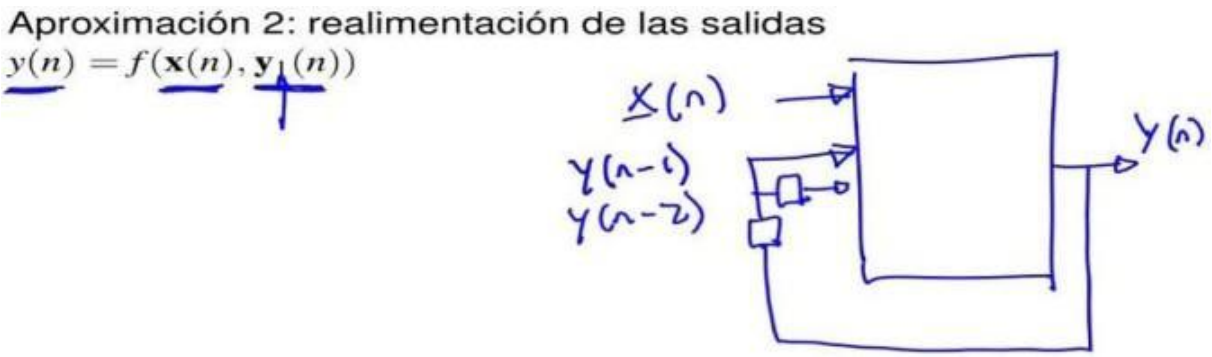
- Hay entradas que no solo contemplan el momento actual sino lo que valieron en instantes anteriores.



- **x** → Varias entradas, una salida (en este caso).
- Incorporamos una **n** = tiempo discreto.
- Planteamos una red <u>neuronal estática</u> cualquiera, y vamos a tener el vector x como entrada, y las entradas desplazadas en los instantes anteriores.

  => tenemos una ENTRADA EXPANDIDA, hasta N:.

  => la red sigue siendo estática, sigue viendo un patrón de entrada cada vez que le pedimos una salida.



Esta segunda aproximación considera las salidas en instantes anteriores, con un retardo, como entradas, aunque también sigue siendo una salida estática.



Esta tercera aproximación considera estados internos de la red neuronal de la capa oculta para alimentar a la red, empezando a tener un comportamiento más dinámico.

Finalmente:

Caso general:

$$y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n), \mathbf{y}_1(n))$$

Considera los 3 tipos anteriores, retroalimentandose con salidas, entradas e intermedios anteriores.

Clasificación de las redes neuronales dinámicas DNN

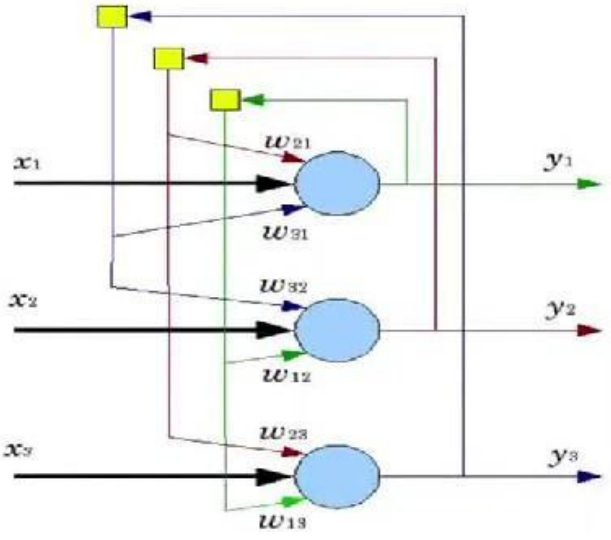
- Redes *con retardo en el tiempo*. (**TDNN**)
  Tienen un comportamiento dinámico, pero las podemos entrenar con los algoritmos que ya conocemos.
- Redes *recurrentes*. (**RNN**)
  Tienen realimentación hacia atrás (la sinapsis puede ir hacia atrás).
  - Redes ***totalmente recurrentes.***
    - ★ Redes de Hopfield (memorias asociativas)
    - ★ Redes de Boltzman (supervisadas)
    - ★ Teoría de la resonancia adaptativa (ART)
  - Redes ***parcialmente recurrentes***. (**PRNN**)
    - ★ Retro propagación a través del tiempo (BPTT)
    - ★ Redes de Elman.
    - ★ Redes de Jordan.

Redes de Hopfield → Redes totalmente recurrentes

- Se hace recurrencia a través de la salida.
  - Pero **no realimenta a la misma neurona de la que sale**, sino a las demás.
- Siempre hay la misma cantidad de entradas, salidas y neuronas.
- No hay capas, solo una.

- Se inicializa con impulsos de entrada, y luego las entradas son las salidas de los instantes anteriores, no vuelven a entrar las entradas originales.
- Tiene función signo → salida +1 y -1
- Los pesos sinápticos son simétricos, y el de la misma neurona es 0.

$$w_{ji} = w_{ij} \quad \forall\, i \neq j$$
$$w_{ii} = 0 \quad \forall i$$



- Modelo Matemático:

$$y_j(n) = sgn(x) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1) - \theta_j\right) \cdots \begin{cases} x > 0 & +1 \\ x = 0 & y_j(n-1) \\ x < 0 & -1 \end{cases}$$

- El comportamiento es bastante parecido al perceptrón, se puede usar la función signo, los pesos se multiplican por la entrada y se suman, hay un umbral o sesgo.
- Pueden tener 3 salidas, pero cuando es 0 la salida es la del instante anterior.

Redes de Hopfield - Generalidades

- Cada neurona tiene un *disparo probabilístico*
- Conexiones *simétricas*
- El entrenamiento es *no-supervisado*
- Pueden utilizarse como *memoria asociativa*

Almacenamiento y recuperación - RdH → Memorias asociativas

Utilizaremos las redes de Hopfield para realizar una memoria asociativa. El proceso de almacenamiento es el entrenamiento de la red y para retomar el contenido solamente se usa o prueba la red.

Almacenamiento (ENTRENAMIENTO)

Dado un conjunto de patrones (memorias fundamentales o datos limpios) x_{k*}:

$$X^* = \{\mathbf{x}_k^* \in \mathbb{R}^N\}$$

El aprendizaje o almacenamiento se denomina Hebbiano:

- Hacemos una especie de promedio de los patrones.

Aprendizaje Hebbiano:

$$w_{ji} = \frac{1}{N} \sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$

- *El peso $\mathbf{w_{ij}}$, que vincula la neurona **i** con la neurona **j**, promedia las entradas de **i** y **j** para todos los patrones, haciendo un producto sobre todos los patrones **k**.*
- ¿Cómo se interpreta esto?:
- Estamos tratando de averiguar qué pasa con la conexión que hay la neurona **i** y la **j**
    - Si ambas siempre son -1 o ambas 1→ el peso se hace positivo y grande
    - Si uno siempre es -1 y otro +1 → el peso se hace negativo y grande
    - Si algunas veces son +1 y -1 o -1 y +1 → el peso va a ser cercano a cero.

  => **Este método está detectando qué neuronas se activan a la vez.**

- **LOS PESOS GUARDAN UNA MEMORIA DE QUÉ TIPO DE RELACIÓN HAY ENTRE LOS PUNTOS/PÍXELES/DIMENSIONES DE LAS ENTRADAS QUE LE VOY MOSTRANDO.**

Observaciones:

- **NO** es un aprendizaje **ITERATIVO.**
- w_{ij} es mayor cuando las neuronas **i** y **j** se tienen que activar juntas (regla de Hebb)
- Hay un límite teórico para la capacidad de almacenamiento:

La cantidad de memorias fundamentales **Pmax** está limitada por la cantidad de neuronas **N**:

$$P_{max} = \frac{N}{2\ln(N)}$$, con un 1% de error.

Prueba (RECUPERACIÓN)

Se le da la misma entrada o una parecida, como la cara de una persona desde otro Angulo y quiero recuperar la imagen limpia.

=> Dado un patrón **x** (incompleto, ruidoso..):

- Inicializar la salida en el instante cero con la entrada:

$$\mathbf{y}(0) = \mathbf{x}$$

- Iteración:
  1. Elegimos al azar alguna de las neuronas:

$$1.\ j^* = rnd(N)$$

  2. Para esa neurona calculamos su salida:

$$2.\ y_{j^*}(n) = sgn\left(\sum_{i=1}^{N} w_{ji} y_i(n-1)\right)$$

  3. Volvemos a 1 hasta no observar cambios en las y

     |

     |

     Una vez que hacemos una pasada completa de todas las neuronas y ninguna cambió => **la red converge a una memoria fundamental.**

Observaciones:

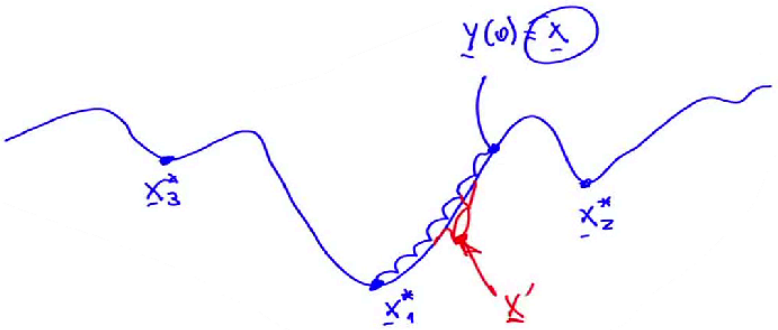
- **ES** un proceso **ITERATIVO** (*dinámico*→ itera hasta estabilizar).
- En general no se utilizan los umbrales θ_j
- Se pueden obtener estados espurios y oscilaciones…*

Estas redes, en el entrenamiento y en la prueba, SON JUSTO LO INVERSO A LAS REDES ESTÁTICAS.

***Campos energéticos de Hopfield**
Se estudia en detalle el caso en que una red queda oscilando, tanto *porque le dimos más memorias fundamentales de las que puede almacenar o porque hay memorias que son muy parecidas.*

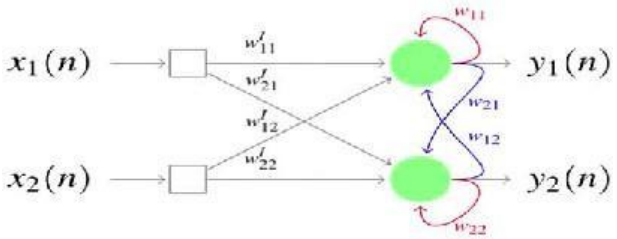
Los campos energéticos son unas distribuciones que podemos observar en una dimensión como zonas de picos y de valles.

- Esa superficie se genera durante el entrenamiento, y, **si todo salió bien**:
  - Las memorias fundamentales se almacenan en los valles.
  - El proceso de recuperación lo que hace es ir bajando, se va acercando hacia una de las fundamentales.
- En caso de que esto **no esté bien hecho**:
  - Puede que se generen valles espudios.
  - Cuando esté bajando caigamos en ellos y veamos una memoria que en realidad no existe.



Retro propagación a través del tiempo (BPTT) → Redes pte recurrentes

- Cualquier cantidad de entradas y salidas, y la arquitectura que sea.
- Hay también pesos sinápticos entre las neuronas, todas con todas, y también consigo mismas.

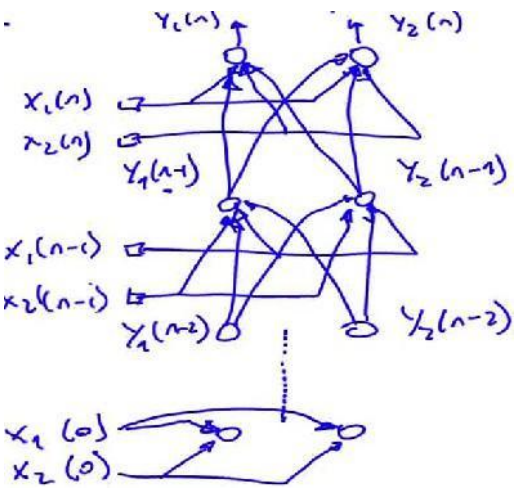


EXPANSIÓN: propagación hacia adelante pura

- En un instante **n**, además de las entradas, a las neuronas van a entrar las salidas de ellas mismas pero de un instante anterior **n-1**.

¿Y cómo se calculan las salidas en el instante **n-1**?

- Podemos pensarlo de la misma manera, expandiendo hasta llegar al instante inicial **n=0**... y obtener:

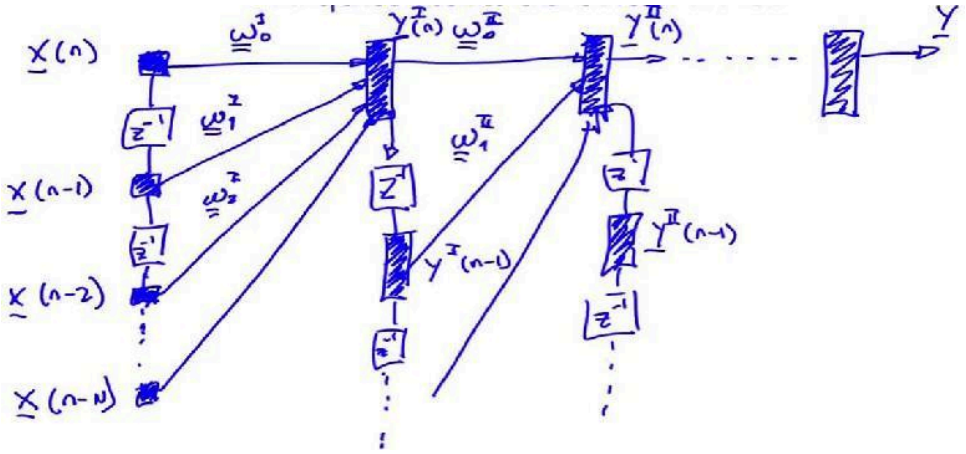


→ Nos quedó una red con muchas capas, pero que <u>ahora es solamente hacia adelante</u> no tiene retroalimentaciones

=> la podemos entrenar por retropropagación como hicimos con el perceptrón multicapa, con ciertos cuidados.

podemos truncar hasta N instantes anteriores.

Redes neuronales con retardos en el tiempo (TDNN)



- Tenemos <u>nuevos conjuntos de pesos</u> que sirven para tener en cuenta la historia de lo que fue pasando en el tiempo/espacio.
- <u>Puede ser entrenada con una retropropagación</u> como el perceptrón multicapa.
- Las primeras capas tienen memoria más inmediata, y a medida que avanzan, las capas van teniendo memoria de más largo plazo, ya que son la memoria de la memoria de la memoria…
  - → esto se conoce como clasificación espacio/temporal.

Arquitecturas de Elman y Jordan

- En **Elman** se considera un retardo de la capa intermedia hacia sí misma, o sea que la salida de la capa intermedia se guarda y el instante siguiente se vuelve a ingresar.
- En **Jordan** la realimentación la toman de la salida.

$y_i \in R$ para $i = 1, 2, \ldots, n$, que representa un punto en el espacio de patrones $P \in R^n$.

-

